

18 Aug React Labs

LAB-11:

Create a React app where a Class Component manages a list of fruits (state + add new fruit), and a Functional Component displays the fruits in a list. Both components should exist in the same file.

Code:

```
import React, { Component } from "react";

// this is a Functional Component to display fruit list

function FruitList({ fruits }) {

  return (

    <div>

      <h3>FruitList</h3>

      <ul>

        {/*here , we iterate through the fruits array and display each fruit as a list item */}

        {fruits.map((fruit, index) => ( // it helps to identify uniquely items

          <li key={index}>{fruit}</li>

        ))}

      </ul>

    </div>

  );

}

//this is a Class Component to manage fruit input and state

class Fruits extends Component {

  constructor(props) {

    super(props);

    this.state = {

      // Here , we have Initialized state with some default fruits and empty newFruit

      fruits: ["Apple", "Banana", "Orange"],

      //this is a controlled state for the input field.

      newFruit: ""

    }

  }

}
```

```

    };
  }

  // here , we Update newFruit state whenever user types in input box

  handleChange = (e) => {

    this.setState({ newFruit: e.target.value });

  };

  //we will add new fruit into fruits array if input is not empty

  addFruit = () => {

    const { newFruit, fruits } = this.state;

    if (newFruit.trim() !== "") { //here , it will prevents adding blank or spaces-only fruit after user type data in input box

      this.setState({

        fruits: [...fruits, newFruit], // Spread operator adds newFruit to array

        newFruit: "" // Clear input box after adding

      });

    }

  };

  render() {

    return (

      <div>

        <h2>Fruits</h2>

        <input

          type="text"

          value={this.state.newFruit}

          onChange={this.handleChange} // this function is used to add fruit into array

          placeholder="Enter fruit"

        />

        <button onClick={this.addFruit}>Add</button>

        { /*here it is used for Passing fruits to functional component */}

        <FruitList fruits={this.state.fruits} />

      </div>

    );

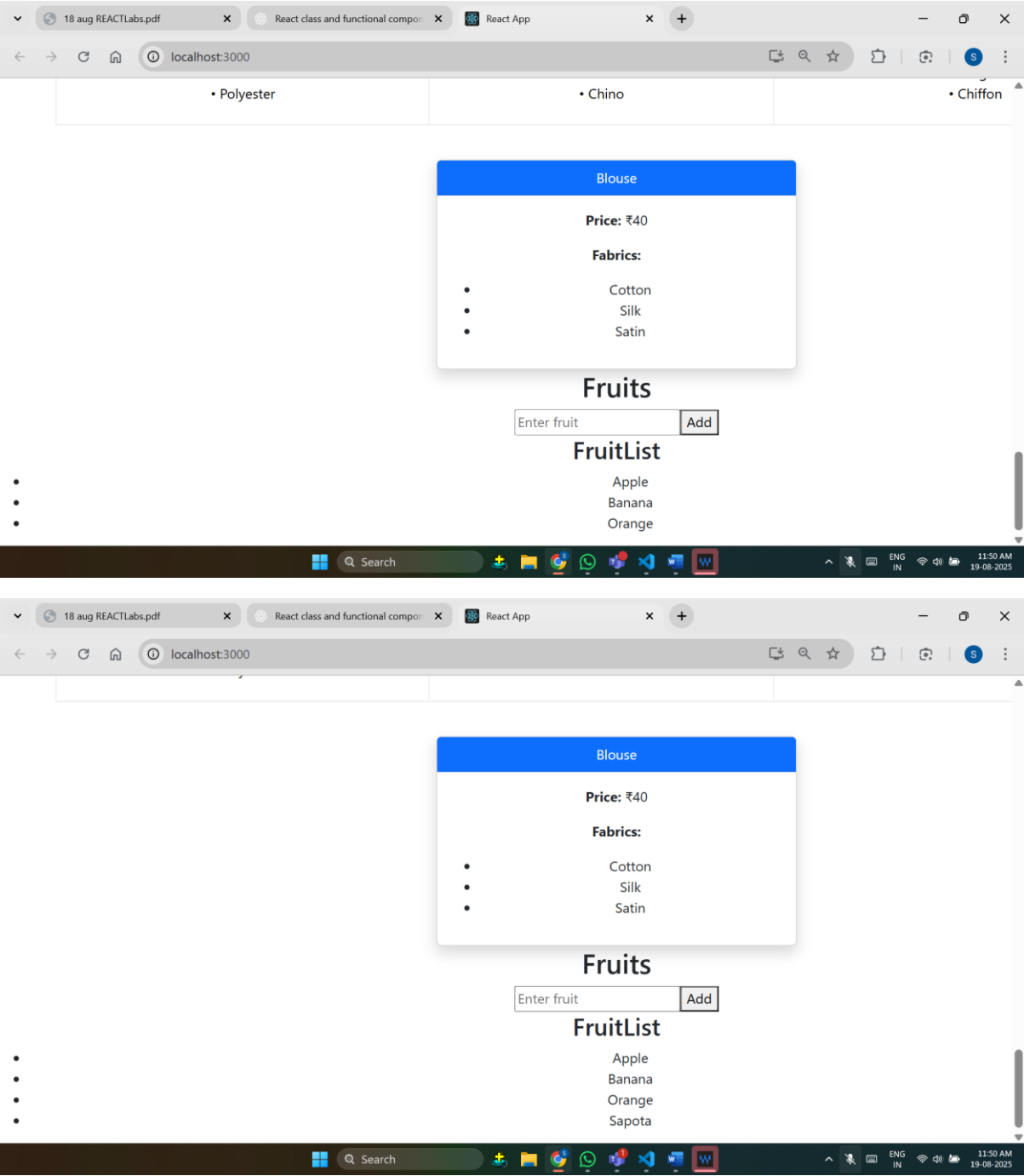
  }

}

```

export default Fruits;

OUTPUT:



Lab-12:

Create a React app where: 1. A Class Component (TelevisionManager) maintains a list of television models in its state.

CODE:

```
import React, { Component } from "react";

// this is a Functional Component to display television models

function TelevisionList({ models }) {

  return (

    <div>

      <h3>Television Models</h3>

      <ul>

        {/*here it iterates through models array and display each model */}

        {models.map((model, index) => (

          <li key={index}>{model}</li>

        ))}

      </ul>

    </div>

  );

}

// Class Component to manage television models

class TelevisionManager extends Component {

  constructor(props) {

    super(props);

    // Initial state with some models and empty input

    this.state = {

      models: ["Samsung QLED", "LG OLED", "Sony Bravia"],

      newModel: ""

    };

  }

  // here we will Update input value in state whenever user types
```

```

handleChange = (e) => {
  this.setState({ newModel: e.target.value });
};

// Add new model to models array using spread operator
addModel = () => {
  const { newModel, models } = this.state;

  if (newModel !== "") { // it is used to prevent empty entries

    this.setState({
      models: [...models, newModel], // here we use spread operator to adds items fro old array to new array item and
it uses to reduce code

      newModel: "" // clear input after adding
    });
  }
};

render() {
  return (
    <div>
      <h2>Television Manager</h2>

      <input
        type="text"
        value={this.state.newModel}
        onChange={this.handleChange}
        placeholder="Enter television model"
      />

      { /* Button to add new model */ }
      <button onClick={this.addModel}>Add</button>

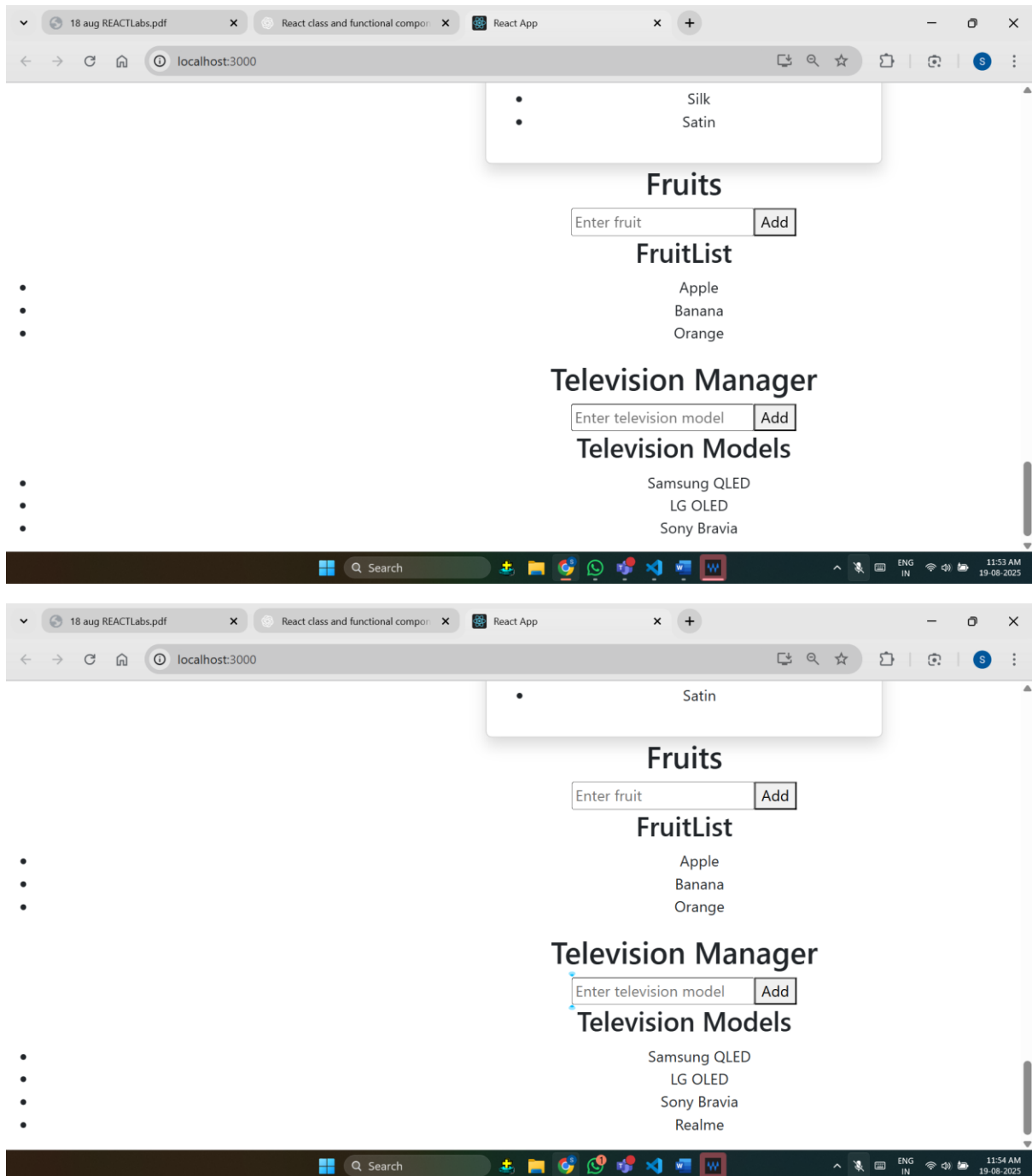
      { /* Pass models as props to functional component */ }
      <TelevisionList models={this.state.models} />
    </div>
  );
}

```

```
}
```

```
export default TelevisionManager;
```

OUTPUT:



LAB-13:

Create a React Class Component called MarriageForm that contains multiple fields (like BrideName, Groom Name, Date, Venue). Use Bootstrap for styling. Manage the form inputs using component state and display the submitted details below the form.

CODE:

```
import React, { Component } from "react";

// this is a Class Component

class MarriageForm extends Component {

  constructor(props) {

    super(props);

    // here , we use state to store form input values and submission flag

    this.state = {

      brideName: "",

      groomName: "",

      date: "",

      venue: "",

      submitted: false // Will be true after clicking submit

    };

  }

  // Handle input box to change (it will updates state for each field present in the form)

  handleChange = (e) => {

    // e.target.name refers to input's name attribute present in form

    // e.target.value is the value entered by user from user

    this.setState({ [e.target.name]: e.target.value });

  };

  // Handle form submit

  handleSubmit = (e) => {

    e.preventDefault(); // Prevents page reload
```

```
this.setState({ submitted: true }); // Set flag to true so details are displayed
};

render() {
  // Destructure values from state
  const { brideName, groomName, date, venue, submitted } = this.state;

  return (
    <div className="container mt-5">
      {/* Form Card */}
      <div className="card p-4 shadow">
        <h2 className="text-center mb-4">Marriage Form</h2>
        <form onSubmit={this.handleSubmit}>
          {/* Bride Name Input box */}
          <div className="form-group mb-3">
            <label>Bride Name</label>
            <input
              type="text"
              className="form-control"
              name="brideName"
              value={brideName}
              onChange={this.handleChange} // updates state on typing in input box
              required
            />
          </div>
          {/* Groom Name Input box */}
          <div className="form-group mb-3">
            <label>Groom Name</label>
            <input
              type="text"
              className="form-control"
              name="groomName"
              value={groomName}
              onChange={this.handleChange}
              required
            />
          </div>
        </form>
      </div>
    </div>
  );
}
```

```
    { /* Marriage Date Input box */}

    <div className="form-group mb-3">

        <label>Marriage Date</label>

        <input

            type="date"

            className="form-control"

            name="date"

            value={date}

            onChange={this.handleChange}

            required

        />

    </div>

    { /* Venue Input box */}

    <div className="form-group mb-3">

        <label>Venue</label>

        <input

            type="text"

            className="form-control"

            name="venue"

            value={venue}

            onChange={this.handleChange}

            required

        />

    </div>

    <button type="submit" className="btn btn-primary w-100">

        Submit

    </button>

</form>

</div>

{ /* Display Submitted Marriage Details */}

{submitted && (

    <div className="card p-4 mt-4 shadow">

        <h3 className="text-center mb-3">Marriage Details</h3>

        <p><strong>Bride Name:</strong> {brideName}</p>
```

```

    <p><strong>Groom Name:</strong> {groomName}</p>

    <p><strong>Date:</strong> {date}</p>

    <p><strong>Venue:</strong> {venue}</p>

  </div>

  )}

</div>

);

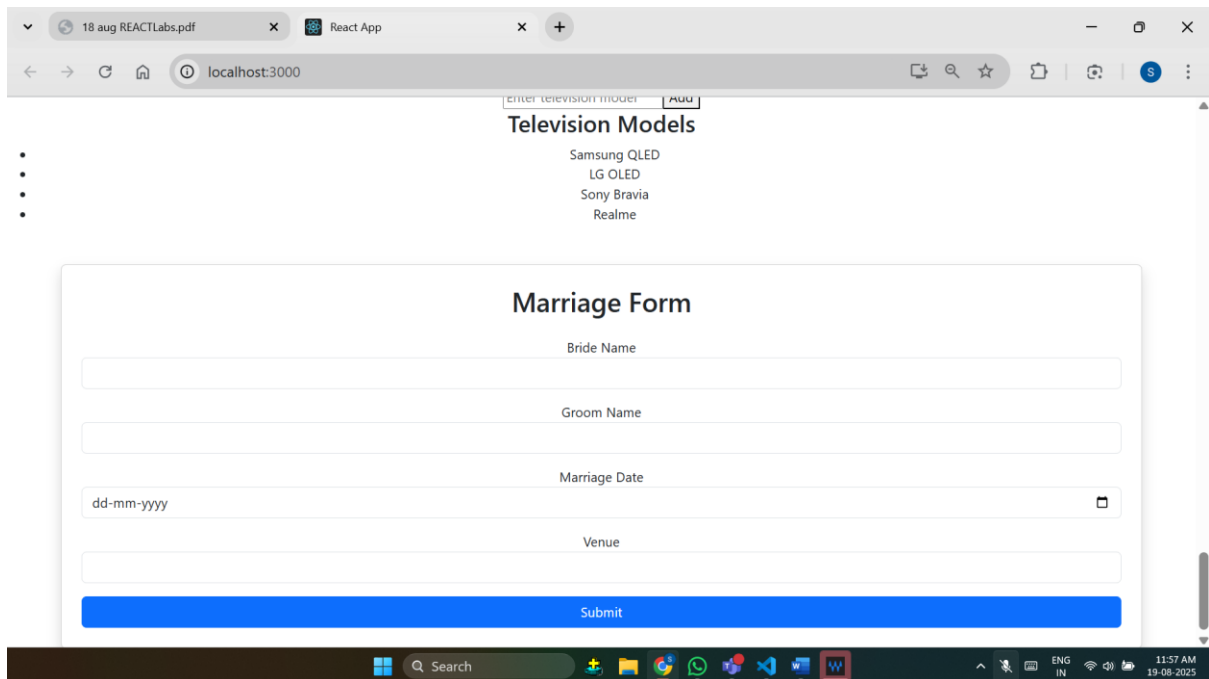
}

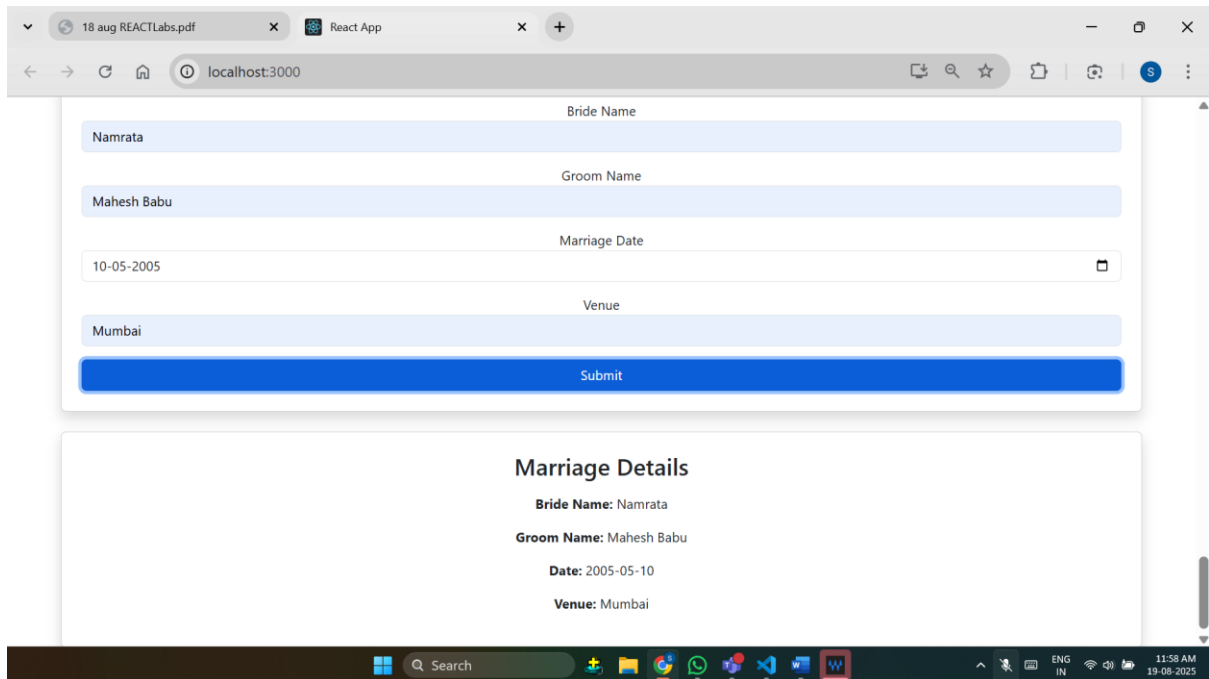
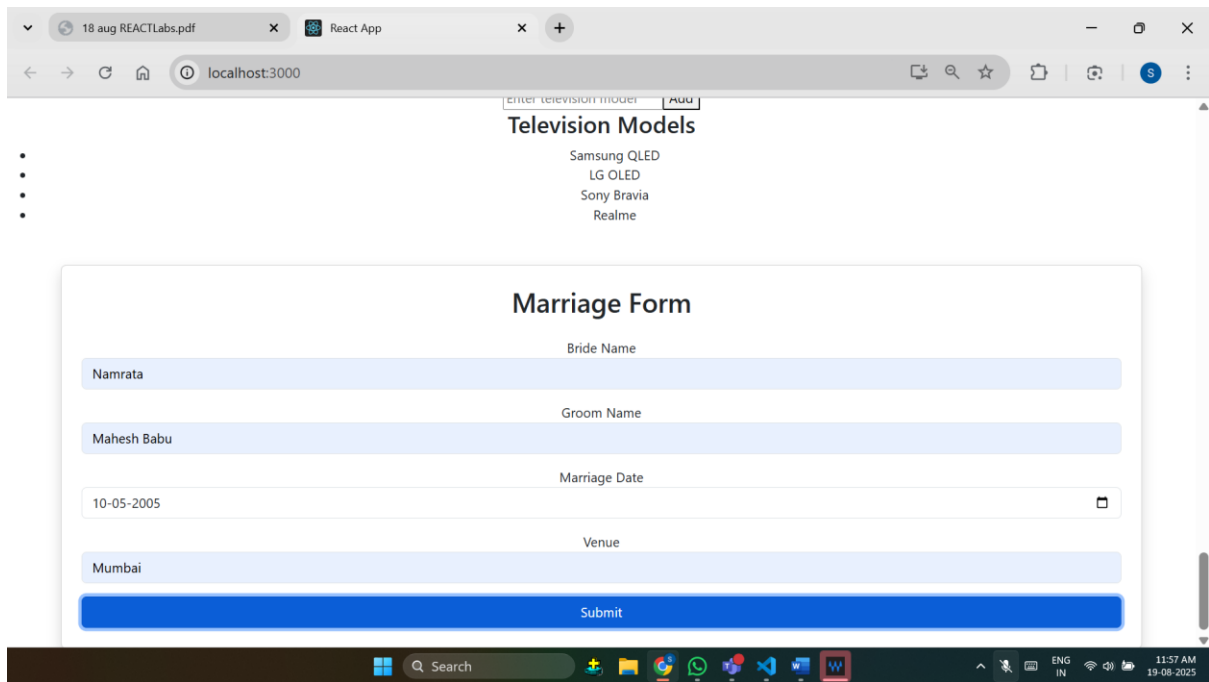
}

export default MarriageForm;

```

OUTPUT:





LAB-14:

Create a React Class Component called AccessoriesForm to demonstrate state management for all types of form controls (text, textarea, radio, checkbox, select). On form submission, display the entered details in a Bootstrap table format.

CODE:

```
import React, { Component } from "react";

//this is a class component which we extends component from react

class AccessoriesForm extends Component {

  constructor(props) {

    super(props);

    // this is a initial state to store form values empty one

    this.state = {

      accessoryName: "",

      description: "",

      category: "",

      brand: "",

      inStock: false,

      warranty: "",

      submittedData: [] // we will store the submitted data here (array of objects)

    };

  }

  // Handle text, textarea, select, number input changes

  handleChange = (e) => {

    const { name, value } = e.target;

    this.setState({ [name]: value }); // Dynamically update state based on input name

  };

  // Handle radio button changes when user selects in form

  handleRadioChange = (e) => {

    this.setState({ brand: e.target.value });

  };

  // Handle checkbox

  handleCheckboxChange = (e) => {

    this.setState({ inStock: e.target.checked });

  };

}
```

```

// Handle form submission

handleSubmit = (e) => {

// it is used to Prevent page reload because if we use action then data

// submitted in using form will be displayed in url

e.preventDefault();

// Collect all data into an object

const formData = {

  accessoryName: this.state.accessoryName,

  description: this.state.description,

  category: this.state.category,

  brand: this.state.brand,

  inStock: this.state.inStock ? "Yes" : "No",

  warranty: this.state.warranty,

};

// here , we will update submittedData array with new form data array

this.setState((prevState) => ({

  submittedData: [...prevState.submittedData, formData], // here we used spread operator to copy data form old
array to new array

  accessoryName: "",

  description: "",

  category: "",

  brand: "",

  inStock: false,

  warranty: "",

}));

});

render() {

return (

<div className="container mt-4">

  <h2 className="mb-4">Accessories Form</h2>

  <form onSubmit={this.handleSubmit} className="mb-4">

    {/* Text Input - Accessory Name */}

    <div className="mb-3">

      <label className="form-label">Accessory Name</label>

```

```
<input
  type="text"
  className="form-control"
  name="accessoryName"
  value={this.state.accessoryName}
  onChange={this.handleChange}
  required
/>
</div>

{/* Textarea - Description */}
<div className="mb-3">
  <label className="form-label">Description</label>
  <textarea
    className="form-control"
    name="description"
    value={this.state.description}
    onChange={this.handleChange}
    required
  />
</div>

{/* Select Dropdown - Category */}
<div className="mb-3">
  <label className="form-label">Category</label>
  <select
    className="form-select"
    name="category"
    value={this.state.category}
    onChange={this.handleChange}
    required>
    <option value="">-- Select Category --</option>
    <option value="Mobile">Mobile</option>
    <option value="Laptop">Laptop</option>
    <option value="Headphones">Headphones</option>
    <option value="Wearables">Wearables</option>
  </select>
```

```
</div>

{/* Radio Buttons - Brand */}

<div className="mb-3">

  <label className="form-label">Brand</label> <br />

  <div className="form-check form-check-inline">

    <input

      type="radio"

      name="brand"

      value="Samsung"

      checked={this.state.brand === "Samsung"}

      onChange={this.handleRadioChange}

      className="form-check-input"

    />

    <label className="form-check-label">Samsung</label>

  </div>

  <div className="form-check form-check-inline">

    <input

      type="radio"

      name="brand"

      value="Apple"

      checked={this.state.brand === "Apple"}

      onChange={this.handleRadioChange}

      className="form-check-input"

    />

    <label className="form-check-label">Apple</label>

  </div>

  <div className="form-check form-check-inline">

    <input

      type="radio"

      name="brand"

      value="Sony"

      checked={this.state.brand === "Sony"}

      onChange={this.handleRadioChange}

      className="form-check-input"

    />
```

```

        <label className="form-check-label">Sony</label>

    </div>

</div>

{/* Checkbox - Availability */}
<div className="mb-3 form-check">

    <input

        type="checkbox"

        className="form-check-input"

        name="inStock"

        checked={this.state.inStock}

        onChange={this.handleChangeCheckbox}

    />

    <label className="form-check-label">Available in Stock</label>

</div>

{/* Number Input - Warranty */}
<div className="mb-3">

    <label className="form-label">Warranty (in years)</label>

    <input

        type="number"

        className="form-control"

        name="warranty"

        value={this.state.warranty}

        onChange={this.handleChange}

        min="0"

        required

    />

</div>

<button type="submit" className="btn btn-primary">

    Submit

</button>

</form>

{/* Bootstrap Table for displaying submitted data in browser */}
{this.state.submittedData.length > 0 && (

    <table className="table table-bordered">

```

```
<thead className="table-dark">

  <tr>

    <th>Accessory Name</th>

    <th>Description</th>

    <th>Category</th>

    <th>Brand</th>

    <th>In Stock</th>

    <th>Warranty</th>

  </tr>

</thead>

<tbody>

  {this.state.submittedData.map((item, index) => (

    <tr key={index}>

      <td>{item.accessoryName}</td>

      <td>{item.description}</td>

      <td>{item.category}</td>

      <td>{item.brand}</td>

      <td>{item.inStock}</td>

      <td>{item.warranty} years</td>

    </tr>

  ))}

</tbody>

</table>

  )}

</div>

);

}

}
```

```
export default AccessoriesForm;
```

OUTPUT:

A screenshot of a web browser displaying a form titled "Accessories Form". The browser's address bar shows "localhost:3000". The form contains the following fields and controls:

- Accessory Name:** A text input field.
- Description:** A text input field.
- Category:** A dropdown menu with the placeholder text "-- Select Category --".
- Brand:** Three radio buttons labeled "Samsung", "Apple", and "Sony".
- Available in Stock:** A checkbox.
- Warranty (in years):** A text input field.
- Submit:** A blue button at the bottom of the form.

The Windows taskbar at the bottom shows the time as 12:02 PM on 19-08-2025.

A screenshot of the same "Accessories Form" in the web browser, now with data entered into the fields:

- Accessory Name:** "Electronics"
- Description:** "Used for office works"
- Category:** "Laptop"
- Brand:** "Samsung" (selected with a blue dot)
- Available in Stock:** Checked (checkbox has a blue checkmark)
- Warranty (in years):** "5"

The "Submit" button remains at the bottom. The browser's address bar still shows "localhost:3000". The Windows taskbar at the bottom shows the time as 12:03 PM on 19-08-2025.

18 aug REACTLabs.pdf

React App

localhost:3000

Accessory Name

Description

Category

-- Select Category --

Brand

☐ Samsung
☐ Apple
☐ Sony

☐ Available in Stock

Warranty (in years)

Submit

Accessory Name	Description	Category	Brand	In Stock	Warranty
Electronics	Used for office works	Laptop	Samsung	Yes	5 years

18 aug REACTLabs.pdf

React App

localhost:3000

Description

Category

-- Select Category --

Brand

☐ Samsung
☐ Apple
☐ Sony

☐ Available in Stock

Warranty (in years)

Submit

Accessory Name	Description	Category	Brand	In Stock	Warranty
Electronics	Used for office works	Laptop	Samsung	Yes	5 years
Electronics	Used for online works	Mobile	Apple	No	3 years

LAB-15:

Problem Statement: Create a React Functional Component that:

1. Manages a form with multiple input fields (for Baking items) using React Hooks (useState). • Example fields: Item Name, Quantity, Ingredients, Baking Time, Category (Cake/Bread/Pastry)
2. On form submission, the details should be stored in a state array (list of baking items).
3. Display all submitted baking items inside a Bootstrap Table.
4. Apply Bootstrap styling (container, card, form-control, btn, table, etc.).

CODE:

```
import React, { useState } from "react";

function BakingItemsForm() {

  // State for form input fields (single object for all fields)

  const [form, setForm] = useState({

    itemName: "",

    quantity: "",

    ingredients: "",

    bakingTime: "",

    category: "Cake", // default category

  });

  // State to store submitted baking items here (array of objects)

  const [items, setItems] = useState([]);

  // Handle change for any input field

  const handleChange = (e) => {

    // Destructure name and value from the event target

    const { name, value } = e.target; // get name & value from event

    // Update the state using functional form of setForm

    setForm((prevForm) => ({

      ...prevForm, // copy old values

      [name]: value, // update only changed field

    }));

  };

}
```

```

// Handle form submission

const handleSubmit = (e) => {

  e.preventDefault(); //it is used to prevent page reload

  setItems((prevItems) => [...prevItems, form]); {/* add form data to items */}

  setForm({

    itemName: "",

    quantity: "",

    ingredients: "",

    bakingTime: "",

    category: "Cake", // reset to default

  });

};

return (

  <div className="container mt-5">

    {/* Card for form */}

    <div className="card shadow p-4">

      <h3 className="mb-3"> 🍰 Baking Items Form</h3>

      <form onSubmit={handleSubmit}>

        {/* Item Name */}

        <div className="mb-3">

          <label className="form-label">Item Name</label>

          <input

            type="text"

            name="itemName"

            value={form.itemName}

            onChange={handleChange}

            className="form-control"

            placeholder="Enter item name"

            required

          />

        </div>

        {/* Quantity */}

        <div className="mb-3">

          <label className="form-label">Quantity</label>

          <input

```

```
    type="number"

    name="quantity"

    value={form.quantity}

    onChange={handleChange}

    className="form-control"

    placeholder="Enter quantity"

    required

  />
</div>

{/* Ingredients */}
<div className="mb-3">

  <label className="form-label">Ingredients</label>

  <textarea

    name="ingredients"

    value={form.ingredients}

    onChange={handleChange}

    className="form-control"

    placeholder="Enter ingredients"

    required

  />

</div>

{/* Baking Time */}
<div className="mb-3">

  <label className="form-label">Baking Time (minutes)</label>

  <input

    type="number"

    name="bakingTime"

    value={form.bakingTime}

    onChange={handleChange}

    className="form-control"

    placeholder="Enter baking time"

    required

  />

</div>
```

```

    { /* Category */

    <div className="mb-3">

        <label className="form-label">Category</label>

        <select

            name="category"

            value={form.category}

            onChange={handleChange}

            className="form-select"

            >

                <option value="Cake">Cake</option>

                <option value="Bread">Bread</option>

                <option value="Pastry">Pastry</option>

            </select>

        </div>

    { /* Submit Button */

    <button type="submit" className="btn btn-success">

        Add Baking Item

    </button>

    </form>

    </div>

    { /* Table to show submitted items */

    {items.length > 0 && (

    <div className="mt-5">

        <h4>Baking Items List</h4>

        <table className="table table-bordered table-striped">

            <thead className="table-dark">

                <tr>

                    <th>#</th>

                    <th>Item Name</th>

                    <th>Quantity</th>

                    <th>Ingredients</th>

                    <th>Baking Time</th>

                    <th>Category</th>

                </tr>

            </thead>

```

```
<tbody>

  {items.map((item, index) => (

    <tr key={index}>

      <td>{index + 1}</td>

      <td>{item.itemName}</td>

      <td>{item.quantity}</td>

      <td>{item.ingredients}</td>

      <td>{item.bakingTime} mins</td>

      <td>{item.category}</td>

    </tr>

  ))}

</tbody>

</table>

</div>

  )}

</div>

);

}

export default BakingItemsForm;
```

OUTPUT:

A screenshot of a web browser displaying a form titled "Baking Items Form". The form is located at `localhost:3000`. It contains several input fields: "Item Name" (placeholder: "Enter item name"), "Quantity" (placeholder: "Enter quantity"), "Ingredients" (placeholder: "Enter ingredients"), "Baking Time (minutes)" (placeholder: "Enter baking time"), and a "Category" dropdown menu (currently showing "Cake"). A green "Add Baking Item" button is at the bottom. The browser's taskbar shows various application icons and the system clock indicates 12:07 PM on 19-08-2025.

A screenshot of the same "Baking Items Form" with data entered into the fields. The "Item Name" field contains "Egg Puff", the "Quantity" field contains "5", the "Ingredients" field contains "Egg,Dough,Masala Powder,Oven", and the "Baking Time (minutes)" field contains "20". The "Category" dropdown menu is now set to "Bread". The "Add Baking Item" button remains at the bottom. The browser's taskbar and system clock are consistent with the previous screenshot.

18 aug REACTLabs.pdfReact Applocalhost:3000

Enter item name

Quantity

Enter quantity

Ingredients

Enter ingredients

Baking Time (minutes)

Enter baking time

Category

Cake

Add Baking Item

Baking Items List

#	Item Name	Quantity	Ingredients	Baking Time	Category
1	Egg Puff	5	Egg,Dough,Masala Powder,Oven	20 mins	Bread

18 aug REACTLabs.pdfReact Applocalhost:3000

Quantity

Enter quantity

Ingredients

Enter ingredients

Baking Time (minutes)

Enter baking time

Category

Cake

Add Baking Item

Baking Items List

#	Item Name	Quantity	Ingredients	Baking Time	Category
1	Egg Puff	5	Egg,Dough,Masala Powder,Oven	20 mins	Bread
2	Butterscoth Cake	2	Butterscoth flavor,cream, bread,oven	50 mins	Cake

LAB-16:

Flight Booking Form (Controlled & Uncontrolled Components)

Create a React application in a single file that demonstrates both Controlled and Uncontrolled components using Bootstrap

CODE:

```
import React, { Component } from "react";

// Controlled Component

class FlightBookingControlled extends Component {

  constructor(props) {

    super(props);

    // state for all form fields as default thisvalues

    this.state = {

      passengerName: "",

      email: "",

      gender: "",

      mealPreference: "",

      specialRequest: "",

      submittedData: [], // stores submitted passenger data

    };

  }

  // handles changes for input fields

  handleChange = (e) => {

    const { name, value } = e.target; // get name & value from input

    this.setState({ [name]: value }); // update state dynamically

  };

  // handles form submission

  handleSubmit = (e) => {

    e.preventDefault(); // it will prevent page reload

    //this is the values taken from state

    const formData = {

      passengerName: this.state.passengerName,

      email: this.state.email,

      gender: this.state.gender,
```

```
mealPreference: this.state.mealPreference,
specialRequest: this.state.specialRequest,
};

// store data in array & reset fields
this.setState((prevState) => ({
  submittedData: [...prevState.submittedData, formData],
  passengerName: "",
  email: "",
  gender: "",
  mealPreference: "",
  specialRequest: "",
}));
};

render() {
  return (
    <div className="col-md-6">
      <h3>Controlled Flight Booking Form</h3>
      <form onSubmit={this.handleSubmit}>
        {/* Passenger Name */}
        <div className="mb-3">
          <label className="form-label">Passenger Name</label>
          <input
            type="text"
            className="form-control"
            name="passengerName"
            value={this.state.passengerName}
            onChange={this.handleChange}
            required
          />
        </div>
        {/* Email */}
        <div className="mb-3">
          <label className="form-label">Email</label>
          <input
            type="email"
```

```

        className="form-control"

        name="email"

        value={this.state.email}

        onChange={this.handleChange}

        required
    />
</div>

{/* Gender */}

<div className="mb-3">

    <label className="form-label">Gender</label><br />

    <input

        type="radio"

        name="gender"

        value="Male"

        checked={this.state.gender === "Male"}

        onChange={this.handleChange}

    /> Male

    <input

        type="radio"

        name="gender"

        value="Female"

        checked={this.state.gender === "Female"}

        onChange={this.handleChange}

        className="ms-3"

    /> Female

</div>

{/* Meal Preference */}

<div className="mb-3">

    <label className="form-label">Meal Preference</label>

    <select

        className="form-select"

        name="mealPreference"

        value={this.state.mealPreference}

        onChange={this.handleChange}

        required

```

```

>

<option value="">--Select--</option>

<option value="Veg">Veg</option>

<option value="Non-Veg">Non-Veg</option>

</select>

</div>

{/* Special Request */}

<div className="mb-3">

  <label className="form-label">Special Request</label>

  <textarea

    className="form-control"

    name="specialRequest"

    value={this.state.specialRequest}

    onChange={this.handleChange}

  />

</div>

<button type="submit" className="btn btn-primary">Submit</button>

</form>

{/* Submitted Data Table */}

{this.state.submittedData.length > 0 && (

  <div className="mt-4">

    <h5>Submitted Details</h5>

    <table className="table table-bordered">

      <thead>

        <tr>

          <th>Passenger Name</th>

          <th>Email</th>

          <th>Gender</th>

          <th>Meal Preference</th>

          <th>Special Request</th>

        </tr>

      </thead>

      <tbody>

        {this.state.submittedData.map((data, index) => { // it used to iterate values from data

          <tr key={index}> {/*unique keys for data */}

```

```

        <td>{data.passengerName}</td>

        <td>{data.email}</td>

        <td>{data.gender}</td>

        <td>{data.mealPreference}</td>

        <td>{data.specialRequest}</td>

    </tr>

    )}}

</tbody>

</table>

</div>

)}

</div>

);

}

}

```

```
export default FlightBookingControlled;
```

```
import React, { Component, createRef } from "react";
```

```
// Uncontrolled Component
```

```
class FlightBookingUncontrolled extends Component {
```

```
  constructor(props) {
```

```
    //we will use ref for uncontrolled component
```

```
    super(props);
```

```
    // create refs for each field
```

```
    this.flightNumberRef = createRef();
```

```
    this.sourceRef = createRef();
```

```
    this.destinationRef = createRef();
```

```
    this.dateRef = createRef();
```

```
    this.termsRef = createRef();
```

```
    this.state = {
```

```
      details: null, // store submitted details
```

```
    };
```

```
  }
```

```

// handle form submission

handleSubmit = (e) => {

  e.preventDefault();

  // here , it will collect values directly using refs

  const details = {

    flightNumber: this.flightNumberRef.current.value,

    source: this.sourceRef.current.value,

    destination: this.destinationRef.current.value,

    travelDate: this.dateRef.current.value,

    termsAccepted: this.termsRef.current.checked,

  };

  this.setState({ details });

};

render() {

  return (

    <div className="col-md-6">

      <h3>Uncontrolled Flight Booking Form</h3>

      <form onSubmit={this.handleSubmit}>

        {/* Flight Number */}

        <div className="mb-3">

          <label className="form-label">Flight Number</label>

          <input type="text" className="form-control" ref={this.flightNumberRef} required />

        </div>

        {/* Source */}

        <div className="mb-3">

          <label className="form-label">Source</label>

          <input type="text" className="form-control" ref={this.sourceRef} required />

        </div>

        {/* Destination */}

        <div className="mb-3">

```

```

<label className="form-label">Destination</label>

<input type="text" className="form-control" ref={this.destinationRef} required />

</div>

{/* Travel Date */}

<div className="mb-3">

  <label className="form-label">Travel Date</label>

  <input type="date" className="form-control" ref={this.dateRef} required />

</div>

{/* Terms Checkbox */}

<div className="mb-3 form-check">

  <input type="checkbox" className="form-check-input" ref={this.termsRef} />

  <label className="form-check-label">Terms Accepted</label>

</div>

<button type="submit" className="btn btn-success">Submit</button>

</form>

{/* Display Submitted Data */}
{this.state.details && (
  <div className="card mt-4">
    <div className="card-body">
      <h5 className="card-title">Flight Details</h5>
      <p><b>Flight Number:</b> {this.state.details.flightNumber}</p>
      <p><b>Source:</b> {this.state.details.source}</p>
      <p><b>Destination:</b> {this.state.details.destination}</p>
      <p><b>Travel Date:</b> {this.state.details.travelDate}</p>
      <p><b>Terms Accepted:</b> {this.state.details.termsAccepted ? "Yes" : "No"}</p>
    </div>
  </div>
)}
</div>

);
}

```

}

export default FlightBookingUncontrolled;

OUTPUT:

The image displays two screenshots of a web application running on localhost:3000. The application features a table of food items and a controlled flight booking form.

Table:

#	Item Name	Quantity	Ingredients	Baking Time	Category
1	Egg Puff	5	Egg,Dough,Masala Powder,Oven	20 mins	Bread
2	Butterscoth Cake	2	Butterscoth flavor,cream, bread,oven	50 mins	Cake

Controlled Flight Booking Form:

Passenger Name

Email

Gender

☐ Male ☐ Female

Meal Preference

--Select--

Special Request

Submit

Second Screenshot (Form Filled):

Passenger Name: Satya Sai

Email: satya@gmail.com

Gender: ☒ Male ☐ Female

Meal Preference: Veg

Special Request: Give veg only

Submit

18 aug REACTLabs.pdf x React App x +

localhost:3000

Controlled Flight Booking Form

Passenger Name

Email

Gender

☐ Male ☐ Female

Meal Preference

--Select--

Special Request

Submit

Submitted Details

Passenger Name	Email	Gender	Meal Preference	Special Request
Satya Sai	satya@gmail.com	Male	Veg	Glve veg only

18 aug REACTLabs.pdf x React App x +

localhost:3000

Submit

Submitted Details

Passenger Name	Email	Gender	Meal Preference	Special Request
Satya Sai	satya@gmail.com	Male	Veg	Glve veg only

Uncontrolled Flight Booking Form

Flight Number

Source

Destination

Travel Date

dd-mm-yyyy

☐ Terms Accepted

Submit

18 aug REACTLabs.pdf

React App

localhost:3000

Submit

Submitted Details

Passenger Name	Email	Gender	Meal Preference	Special Request
Satya Sai	satya@gmail.com	Male	Veg	Give veg only

Uncontrolled Flight Booking Form

Flight Number

SAP984

Source

Hyderabad

Destination

Shiridi

Travel Date

19-09-2025

☐ Terms Accepted

Submit

18 aug REACTLabs.pdf

React App

localhost:3000

SAP984

Source

Hyderabad

Destination

Shiridi

Travel Date

19-09-2025

☐ Terms Accepted

Submit

Flight Details

Flight Number: SAP984

Source: Hyderabad

Destination: Shiridi

Travel Date: 2025-09-19

Terms Accepted: No

LAB-17:

Objective: Create a React Class Component that allows users to add a movie to a list. The component should include multiple movie fields and demonstrate all class component lifecycle methods in action

CODE:

```
import React, { Component } from "react";

// this is a class component

class MovieForm extends Component {

  constructor(props) {

    super(props);

    console.log("constructor executed");

    //this is initial state form values

    this.state = {

      title: "",

      director: "",

      releaseYear: "",

      genre: "Action",

      rating: "",

      description: "",

      platforms: [],

      movies: [],

      showForm: true, // toggle form to demonstrate componentWillUnmount

    };

  }

  // Called before render when props/state changes

  static getDerivedStateFromProps(props, state) {

    console.log("getDerivedStateFromProps executed");

    return null; // No state change from props

  }

  // Called once after the component is mounted (rendered on the screen)

  componentDidMount() {

    console.log("componentDidMount executed");
```

```
}
```

```
// Called before re-rendering when props/state change
```

```
// Decide whether the component should re-render (return true/false)
```

```
shouldComponentUpdate(nextProps, nextState) {
```

```
  console.log("shouldComponentUpdate executed");
```

```
  return true; // returning true means allow re-render
```

```
}
```

```
// Called right before the DOM updates
```

```
// Gives you a chance to capture some information
```

```
getSnapshotBeforeUpdate(prevProps, prevState) {
```

```
  console.log("getSnapshotBeforeUpdate executed");
```

```
  return null; // return value will be passed to componentDidUpdate
```

```
}
```

```
// Called immediately after updating (re-render)
```

```
// Useful for making network requests or DOM manipulations
```

```
componentDidUpdate(prevProps, prevState, snapshot) {
```

```
  console.log("componentDidUpdate executed");
```

```
}
```

```
// Called right before the component is removed from the DOM
```

```
// Useful for cleanup (e.g., clear timers, cancel API calls, remove event listeners)
```

```
componentWillUnmount() {
```

```
  console.log("componentWillUnmount executed");
```

```
}
```

```
// handle input changes
```

```
handleChange = (e) => {
```

```
  const { name, value, type, checked } = e.target;
```

```
  if (type === "checkbox") {
```

```
    this.setState((prevState) => {
```

```
      if (checked) {
```

```
        return { platforms: [...prevState.platforms, value] };
```

```
      } else {
```

```
        // by using filter we can copy only the elements except value
```

```
        return { platforms: prevState.platforms.filter((p) => p !== value) };
```

```

    }
  });
} else {
  this.setState({ [name]: value });
}
};

```

```

// handle form submission

```

```

handleSubmit = (e) => {
  e.preventDefault();

  //destructure the values

  const { title, director, releaseYear, genre, rating, description, platforms } = this.state;

  const newMovie = {

    title,

    director,

    releaseYear,

    genre,

    rating,

    description,

    platforms,

  };

  this.setState((prevState) => ({

    movies: [...prevState.movies, newMovie], //spread operator is used to copy old array to new array

    title: "",

    director: "",

    releaseYear: "",

    genre: "Action",

    rating: "",

    description: "",

    platforms: [],

  }));
};

```

```

// Method to toggle the form's visibility

```

```

toggleForm = () => {

  // setState with a callback function to ensure we use the latest state

```

```
this.setState((prevState) => ({
  // Flip the boolean value: if true → false, if false → true
  showForm: !prevState.showForm
}));
};

render() {
  console.log("render executed");
  return (
    <div className="container mt-4">
      <button className="btn btn-warning mb-3" onClick={this.toggleForm}>
        {this.state.showForm ? "Unmount Form" : "Mount Form"}
      </button>
      {this.state.showForm && (
        <div className="card p-4">
          <h3 className="mb-3">Add Movie</h3>
          <form onSubmit={this.handleSubmit}>
            {/* Movie Title */}
            <div className="mb-3">
              <label className="form-label">Movie Title</label>
              <input
                type="text"
                className="form-control"
                name="title"
                value={this.state.title}
                onChange={this.handleChange}
                required
              />
            </div>
            {/* Director */}
            <div className="mb-3">
              <label className="form-label">Director</label>
              <input
                type="text"
                className="form-control"
                name="director"
```

```
        value={this.state.director}

        onChange={this.handleChange}

        required

      />
    </div>

    { /* Release Year */}

    <div className="mb-3">

      <label className="form-label">Release Year</label>

      <input

        type="number"

        className="form-control"

        name="releaseYear"

        value={this.state.releaseYear}

        onChange={this.handleChange}

        required

      />

    </div>

    { /* Genre */}

    <div className="mb-3">

      <label className="form-label">Genre</label>

      <select

        className="form-select"

        name="genre"

        value={this.state.genre}

        onChange={this.handleChange}

        required

      >

        <option value="Action">Action</option>

        <option value="Comedy">Comedy</option>

        <option value="Drama">Drama</option>

        <option value="Sci-Fi">Sci-Fi</option>

        <option value="Horror">Horror</option>

      </select>

    </div>
```

```

    { /* Rating */ }

    <div className="mb-3">

      <label className="form-label">Rating</label>

      <br />

      {[1, 2, 3, 4, 5].map((num) => (

        <label key={num} className="me-2">

          <input

            type="radio"

            name="rating"

            value={num}

            checked={this.state.rating === String(num)}

            onChange={this.handleChange}

            className="me-1"

          />

          {num}

        </label>

      )]}

    </div>

    { /* Description */ }

    <div className="mb-3">

      <label className="form-label">Description</label>

      <textarea

        className="form-control"

        name="description"

        value={this.state.description}

        onChange={this.handleChange}

      />

    </div>

    { /* Platforms */ }

    <div className="mb-3">

      <label className="form-label">Available on Streaming Platforms</label>

      <br />

      [{"Netflix", "Amazon Prime", "Disney+", "Others"}].map((platform) => (

        <label key={platform} className="me-3">

          <input

```

```
        type="checkbox"
        value={platform}
        checked={this.state.platforms.includes(platform)}
        onChange={this.handleChange}
        className="me-1"
      />
      {platform}
    </label>
  )))
</div>
```

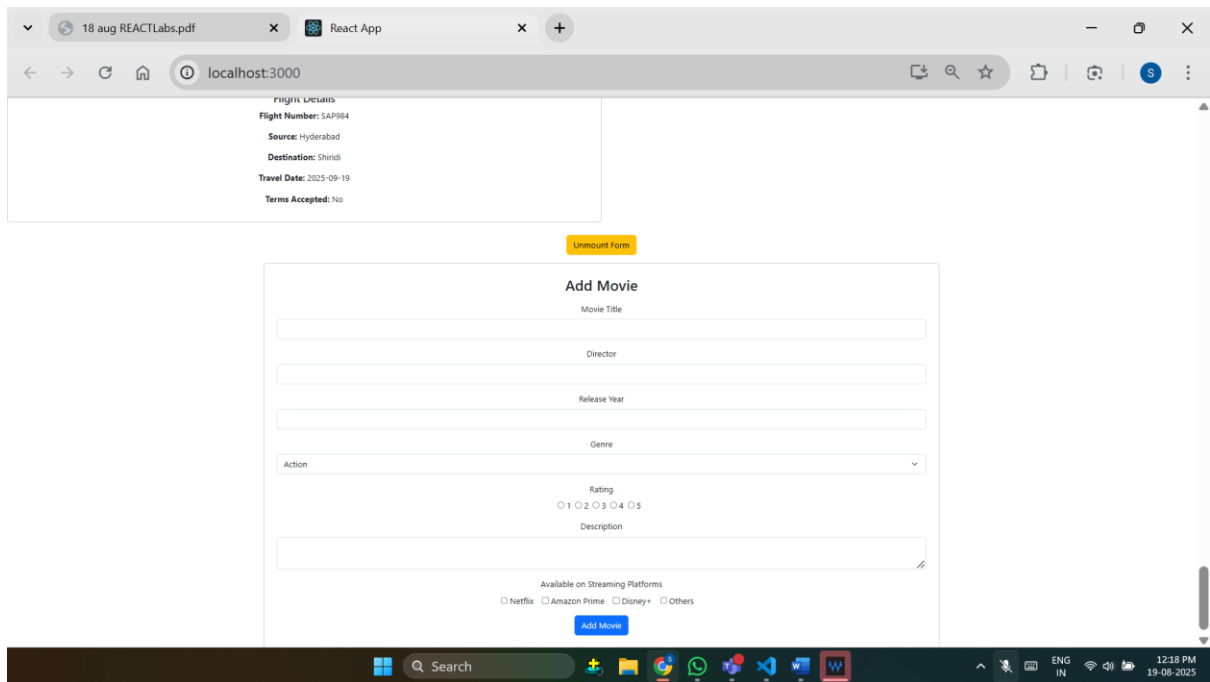
```
    <button type="submit" className="btn btn-primary">
      Add Movie
    </button>
  </form>
</div>
})
```

```
{/* Movies Table */}
{this.state.movies.length > 0 && (
  <div className="mt-4">
    <h4>Movie List</h4>
    <table className="table table-bordered">
      <thead>
        <tr>
          <th>Title</th>
          <th>Director</th>
          <th>Release Year</th>
          <th>Genre</th>
          <th>Rating</th>
          <th>Description</th>
          <th>Available on</th>
        </tr>
      </thead>
      <tbody>
```

```
{this.state.movies.map((movie, index) => (  
  <tr key={index}>  
    <td>{movie.title}</td>  
    <td>{movie.director}</td>  
    <td>{movie.releaseYear}</td>  
    <td>{movie.genre}</td>  
    <td>{movie.rating}</td>  
    <td>{movie.description}</td>  
    <td>{movie.platforms.join(", ")}</td>  
  </tr>  
  )}  
</tbody>  
</table>  
</div>  
  )}  
</div>  
);  
}  
}
```

```
export default MovieForm;
```

OUTPUT:



18 aug REACTLabs.pdf

React App

localhost:3000

Flight Details

Flight Number: SAP984

Source: Hyderabad

Destination: Shindi

Travel Date: 2025-09-19

Terms Accepted: No

Unmount Form

Add Movie

Movie Title

aharshi

Director

Vamsi Paidipally

Release Year

2019

Genre

Action

Rating

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

Description

Action genre

Available on Streaming Platforms

☐ Netflix ☒ Amazon Prime ☐ Disney+ ☒ Others

Add Movie

18 aug REACTLabs.pdf

React App

localhost:3000

Terms Accepted: No

Unmount Form

Add Movie

Movie Title

Director

Release Year

Genre

Action

Rating

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

Description

Available on Streaming Platforms

☐ Netflix ☐ Amazon Prime ☐ Disney+ ☐ Others

Add Movie

Movie List

Title	Director	Release Year	Genre	Rating	Description	Available on
aharshi	Vamsi Paidipally	2019	Action	5	Action genre	Amazon Prime, Others

18 aug REACTLabs.pdf

React App

localhost:3000

Satya Sai

satya@gmail.com

Male

Veg

Give veg only

Uncontrolled Flight Booking Form

Flight Number

Source

Destination

Travel Date

☐ Terms Accepted

Flight Details

Flight Number: SAP984

Source: Hyderabad

Destination: Shirdi

Travel Date: 2025-09-19

Terms Accepted: No

Mount Form

Title	Director	Release Year	Genre	Rating	Description	Available on
aharshi	Vamsi Paidipally	2019	Action	5	Action genre	Amazon Prime, Others

18 aug REACTLabs.pdf

React App

localhost:3000

Name	Email	Gender	Preference	Request
Satya Sai	satya@gmail.com	Male	Veg	Give veg only

Uncontrolled Flight Booking Form

Flight Number

Source

Destination

Travel Date

☐ Terms Accepted

Flight Details

Flight Number: SAP984

Source: Hyderabad

Destination: Shirdi

Travel Date: 2025-09-19

Terms Accepted: No

Mount Form

Title	Director	Release Year	Genre	Rating	Description	Available on
aharshi	Vamsi Paidipally	2019	Action	5	Action genre	Amazon Prime, Others

Elements

Console

Sources

Network

Performance

top

Filter

Default levels

43 Issues

constructor executed

getDerivedStateFromProps executed

render executed

componentDidMount executed

componentWillUnmount executed

componentDidMount executed

Button visibility updated: > Object

getDerivedStateFromProps executed

shouldComponentUpdate executed

render executed

getSnapshotBeforeUpdate executed

componentDidUpdate executed

getDerivedStateFromProps executed

shouldComponentUpdate executed

render executed

getSnapshotBeforeUpdate executed

componentDidUpdate executed

getDerivedStateFromProps executed

shouldComponentUpdate executed

render executed

getSnapshotBeforeUpdate executed

MovieForm.jsx:98

MovieForm.jsx:109

MovieForm.jsx:36

MovieForm.jsx:36

MovieForm.jsx:32

MovieForm.jsx:32

MovieForm.jsx:7

uiManager.js:1

MovieForm.jsx:109

MovieForm.jsx:13

MovieForm.jsx:36

MovieForm.jsx:20

MovieForm.jsx:27

MovieForm.jsx:109

MovieForm.jsx:13

MovieForm.jsx:36

MovieForm.jsx:20

MovieForm.jsx:27

MovieForm.jsx:109

MovieForm.jsx:13

MovieForm.jsx:36

MovieForm.jsx:20

Console

What's new

AI assistance

18 aug REACTLabs.pdf React App localhost:3000

Name	Email	Gender	Preference	Request
Satya Sai	satya@gmail.com	Male	Veg	Give veg only

Uncontrolled Flight Booking Form

Flight Number: SAP984

Source: Hyderabad

Destination: Shirdi

Travel Date: 19-09-2025

☐ Terms Accepted

Submit

Flight Details

Flight Number: SAP984

Source: Hyderabad

Destination: Shirdi

Travel Date: 2025-09-19

Terms Accepted: No

Mount Form

Title	Director	Release Year	Genre	Rating	Description	Available on
-------	----------	--------------	-------	--------	-------------	--------------

Console: 43 Issues

18 aug REACTLabs.pdf React App localhost:3000

Name	Email	Gender	Preference	Request
Satya Sai	satya@gmail.com	Male	Veg	Give veg only

Uncontrolled Flight Booking Form

Flight Number: SAP984

Source: Hyderabad

Destination: Shirdi

Travel Date: 19-09-2025

☐ Terms Accepted

Submit

Flight Details

Flight Number: SAP984

Source: Hyderabad

Destination: Shirdi

Travel Date: 2025-09-19

Terms Accepted: No

Mount Form

Title	Director	Release Year	Genre	Rating	Description	Available on
-------	----------	--------------	-------	--------	-------------	--------------

Console: 43 Issues